

代码风格与格式规范

行宽

每行最多80列。超过80列的行必须换行。在逻辑断点处换行：

```
/* 正面示例 - 在80列处换行 */
int result = very_long_function_name(
    first_parameter,
    second_parameter,
    third_parameter);

/* 正面示例 - 条件表达式换行 */
if (status == STATUS_READY &&
    buffer_count > MIN_BUFFER_COUNT &&
    timeout_remaining > 0) {
    process_data();
}

/* 反面示例 - 超过80列 */
int result = very_long_function_name(first_parameter, second_parameter, third_
```

函数长度

每个函数最多80行。如果函数接近此限制，提取子函数：

```
/* 反面示例 - 单体函数 */
int process_packet(Packet_t *pkt)
{
    /* 120行混合了验证、解析、处理和响应构建 */
}

/* 正面示例 - 分解 */
int process_packet(Packet_t *pkt)
{
    int ret;
```

```

    ret = validate_packet_header(pkt);
    if (ret  $\neq$  ERR_OK) {
        return ret;
    }

    ret = parse_packet_payload(pkt);
    if (ret  $\neq$  ERR_OK) {
        return ret;
    }

    return build_response(pkt);
}

```

嵌套深度

最多4层嵌套。使用提前返回、保护子句或提取函数来减少嵌套深度：

```

/* 反面示例 - 5层嵌套 */
void process(Data_t *d)
{
    if (d  $\neq$  NULL) {
        if (d->valid) {
            for (int i = 0; i < d->count; i++) {
                if (d->items[i].active) {
                    if (d->items[i].value > 0) {
                        /* 第5层 - 太深了 */
                    }
                }
            }
        }
    }
}

/* 正面示例 - 提前返回减少嵌套 */
void process(Data_t *d)
{
    if (d == NULL || !d->valid) {
        return;
    }

    for (int i = 0; i < d->count; i++) {
        process_item(&d->items[i]);
    }
}

```

```
}

static void process_item(Item_t *item)
{
    if (!item->active || item->value ≤ 0) {
        return;
    }
    /* 在合理的嵌套深度中处理 */
}
```

魔法数字

所有数字字面量（除0、1和明显的布尔类值外）必须定义为宏：

```
/* 反面示例 */
if (retry_count > 3) { ... }
uint8_t buffer[256];
timeout = 5000;

/* 正面示例 */
#define MAX_RETRY_COUNT      (3U)
#define RX_BUFFER_SIZE      (256U)
#define DEFAULT_TIMEOUT_MS  (5000U)

if (retry_count > MAX_RETRY_COUNT) { ... }
uint8_t buffer[RX_BUFFER_SIZE];
timeout = DEFAULT_TIMEOUT_MS;
```

宏定义风格

- 值用括号包裹： `#define F00 (42U)`
- 无符号常量使用 `U` 后缀
- 大型无符号常量使用 `UL` 或 `ULL` 后缀
- 相关宏使用统一前缀分组

注释

语言

推荐使用英文注释，或与项目约定一致。

作者署名

创建新文件或主要新章节时：

```
/**
 * @file    uart_driver.c
 * @brief   UART driver with non-blocking async API.
 * @author  兆鸣嵌入式
 */
```

注释质量规则

1. 注释解释"为什么"，而非"是什么"
2. 注释必须与代码一致——修改代码时同步更新注释
3. 删除被注释掉的代码；使用版本控制来保存历史
4. 不写重复代码内容的冗余注释

```
/* 反面示例 - 重复代码内容 */
i++; /* 递增 i */

/* 反面示例 - 代码修改后注释过时 */
/* 发送3次重试 */
for (int i = 0; i < MAX_RETRY_COUNT; i++) { /* 原来是3，现在是5 */

/* 正面示例 - 解释意图 */
/* 使用指数退避重试，避免在错误恢复期间
 * 造成总线拥塞 */
for (int i = 0; i < MAX_RETRY_COUNT; i++) {
```

函数文档

公共函数必须有文档注释：

```
/**
 * @brief   通过驱动异步发送数据。
 * @param   self    驱动实例（不能为NULL）
 * @param   data    数据缓冲区指针
 * @param   len     要发送的字节数（最大 TX_BUF_SIZE）
 * @return  成功返回 ERR_OK，忙碌时返回 ERR_QUEUE_FULL
 * @note    非阻塞。结果通过回调传递。
 *          线程安全：可从任何上下文调用。
 */
```

```
int Driver_SendAsync(Driver_t *self,
                    const uint8_t *data,
                    uint16_t len);
```

死代码

所有未使用的、废弃的或被注释掉的代码必须删除。版本控制会保存历史。死代码的危害：

- 造成对活跃代码的混淆
- 可能被意外启用
- 增加维护负担
- 在代码审查中掩盖真正的问题

const正确性

尽可能使用 `const` 来防止意外修改并文档化意图。这是硬性规则，不是建议。

1. 不被修改的指针参数： `const uint8_t *data`
2. 查找表和配置： `static const`
3. 初始化后不可变的结构体成员

注意： 不要对值参数加 `const`（如 `const uint16_t len`），因为值参数是调用者的副本，加 `const` 没有实际意义。

```
/* 指针参数加 const 表示不修改指向的数据 */
int Protocol_Send(Protocol_t *self,
                 const uint8_t *data,
                 uint16_t len);

/* const 查找表 */
static const uint16_t CRC_TABLE[CRC_TABLE_SIZE] = {
    /* ... */
};
```

static限制作用域

所有不属于模块公共API的函数和变量必须声明为 `static`。无例外。

- `static` 函数：内部辅助函数、硬件访问封装

- `static` 变量：模块私有状态、缓冲区
- 文件级静态变量加 `s_` 前缀：`static int s_count;`
- 真正的全局变量加 `g_` 前缀：`extern int g_system_mode;`

头文件包含

头文件保护

每个头文件必须有头文件保护宏：

```
#ifndef MODULE_NAME_H
#define MODULE_NAME_H

/* ... 内容 ... */

#endif /* MODULE_NAME_H */
```

自包含头文件

每个头文件必须能独立编译。包含它所引用的所有类型 —— 不要依赖引用者来提供。

路径约定

在添加 `#include` 之前，搜索项目中其他文件是如何包含 同一头文件的。使用完全相同的路径格式：

```
/* 如果项目中其他文件这样写： */
#include "drivers/uart_driver.h"

/* 那么你也必须这样写： */
#include "drivers/uart_driver.h"

/* 不要写成： */
#include "uart_driver.h"
#include "../drivers/uart_driver.h"
```

包含顺序

遵循项目现有的包含顺序。常见约定：

```
/* 1. 对应头文件（用于 .c 文件） */
#include "my_module.h"

/* 2. 平台/RTOS头文件 */
#include "FreeRTOS.h"
#include "task.h"

/* 3. HAL/BSP头文件 */
#include "stm32f4xx_hal.h"

/* 4. 项目模块头文件 */
#include "drivers/uart_driver.h"
#include "app/protocol.h"

/* 5. 标准库（如使用） */
#include <string.h>
#include <stdint.h>
```

命名约定

遵循项目现有的命名约定。常见的嵌入式C模式：

元素	约定	示例
宏	全大写蛇形	MAX_RETRY_COUNT
类型	PascalCase + _t	UartDriver_t
函数	模块_动作	UartDriver_Send
局部变量	小写蛇形	retry_count
结构体成员	小写蛇形	buf_size
枚举值	前缀_名称	UART_ERR_TIMEOUT

始终先检查项目的实际约定并保持一致。

函数参数数量

每个函数最多5个参数。需要更多参数时，将相关参数组合成配置结构体。详细的模式和示例参见 03_Clean_Code.md 。

安全编码基本规则

以下规则参考 BARR-C 嵌入式编码标准。

大括号规则

所有 `if / for / while / do-while` 语句必须使用大括号，即使只有一行：

```
/* 禁止 */
if (error) return;

/* 必须 */
if (error) {
    return;
}
```

固定宽度整数类型

禁止使用 `int / short / long / unsigned` 等原生类型，必须使用 `stdint.h` 的固定宽度类型：

```
/* 禁止 */
int count;
unsigned char data;

/* 必须 */
int32_t count;
uint8_t data;
```

位操作安全

- 位操作仅用于无符号整数类型
- 移位量不得大于或等于操作数的位宽
- 有符号整数禁止左移（C标准中是未定义行为）

```
/* 禁止 - 有符号左移是未定义行为 */
int32_t val = 1;
int32_t result = val << 16;

/* 必须 - 使用无符号类型 */
```

```
uint32_t val = 1U;
uint32_t result = val << 16U;
```

返回值检查

非 `void` 函数的返回值不得被丢弃，必须检查或显式标注忽略：

```
/* 禁止 - 返回值被丢弃 */
platform_device_open(dev, DEVICE_FLAG_RDWR);

/* 必须 */
err_t ret = platform_device_open(dev, DEVICE_FLAG_RDWR);
if (ret != ERR_OK) {
    /* 处理错误 */
}
```

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版